GROUPE 6 / PROJET ACADEMIQUE

Rapport technique
Gestion Ecole

Conception et realisation d une plateforme web de gestion scolaire

Projet	Gestion Ecole
Equipe	Groupe 6
Architecture	Frontend React / API Laravel
Environnement	MySQL - Laragon - Sanctum

DAMASSOH Armel - AHOTON Lydia - AHOUANSSOU Fany

HOUKADJI Maxime - AIMONTCHE Geraud

Rapport technique — Gestion École

1. Identification du projet

Élément	Information
Groupe	Groupe 6
Projet	Gestion École
Dépôt GitHub	https://github.com/Armel1617/gestion-ecole
Branche principale	`main`
Architecture	Frontend React/Vite + API backend Laravel
Base de données	MySQL en environnement Laragon

Membres du groupe

- DAMASSOH Armel
- AHOTON Lydia
- AHOUANSSOU Fany
- HOUKADJI Maxime
- AIMONTCHE Géraud

2. Présentation générale

Gestion École est une application web de gestion d'un établissement scolaire.

Elle centralise les étudiants, formations, inscriptions, paiements, notes,

présences, documents et notifications.

L'application est organisée en deux parties :

- Frontend : interface React responsive destinée aux utilisateurs.
- Backend : API Laravel sécurisée qui gère l'authentification, les rôles,

les règles métier et la persistance des données.

Le backend est intégré au dépôt dans le dossier `backend/`.

3. Technologies utilisées

Frontend

- React 19 et Vite
- React Router pour la navigation
- Axios pour les appels API
- Recharts pour les graphiques du tableau de bord
- Lucide React pour les icônes
- Oxlint pour l'analyse du code

Backend

- PHP 8.3 ou supérieur
- Laravel 13
- Laravel Sanctum pour les jetons d'authentification
- Eloquent ORM

- MySQL
- PHPUnit pour les tests
- Laravel Pint pour le formatage PHP
- Queue Laravel pour les notifications différées

4. Travail réalisé sur le frontend

4.1 Authentification et navigation

- Page de connexion et création de compte public étudiant.
- Stockage du jeton d'authentification dans le navigateur.
- Ajout automatique du jeton `Bearer` dans les requêtes Axios.
- Déconnexion automatique lorsque le jeton expire.
- Protection des routes privées.
- Redirection des étudiants vers leur espace personnel.
- Normalisation du rôle `directeur` vers `direction`.
- Navigation adaptée au rôle de l'utilisateur.
- Menu latéral escamotable avec bouton hamburger.
- Page d'accès interdit pour les rôles non autorisés.

4.2 Tableau de bord

- Statistiques des étudiants, formations et inscriptions.
- Revenus estimés et inscriptions actives.
- Évolution mensuelle et formations populaires.
- Gestion des réponses incomplètes pour éviter les écrans blancs.
- Affichage squelette pendant le chargement.

4.3 Étudiants, formations et inscriptions

- Liste paginée des étudiants.
- Recherche par nom, prénom ou adresse e-mail.
- Filtre par formation.
- Création, modification et suppression selon les permissions.
- Gestion des formations.
- Création d'une inscription avec sa date et son statut.
- Messages d'erreur explicites lorsque Laravel ou MySQL est indisponible.

4.4 Paiements

- Liste et création des paiements.
- Association à une inscription.
- Gestion du montant, de la date, de la référence, du statut et des notes.
- Gestion des erreurs de validation Laravel.

4.5 Notes et présences

- Formulaire de création d'une note avec matière, valeur, barème, coefficient, date et commentaire.
- Formulaire de création d'une présence avec date, statut, retard et motif.
- Les statuts disponibles sont `present`, `absent`, `retard` et `excuse`.
- Les formulaires utilisent les inscriptions existantes et les validations du backend.

4.6 Notifications

- Cloche visible dans la barre supérieure.

- Consultation rapide sans quitter la page courante.
- Nombre de notifications non lues.
- Bouton « Tout lire ».
- Actualisation périodique.
- Lien vers la page complète selon le rôle de l'utilisateur.

4.7 Performances et expérience utilisateur

- Chargement différé des pages avec `React.lazy`.
- Réduction du bundle JavaScript initial.
- Chargement non bloquant des polices externes.
- États de chargement et messages d'erreur.
- Interface responsive pour ordinateur et téléphone.

4.8 Organisation technique du frontend

Le point d'entrée `src/main.jsx` installe le mode strict de React, le routeur `BrowserRouter`, puis le contexte global `AuthProvider`. Cette organisation permet à toutes les pages de partager l'utilisateur connecté et le jeton Sanctum sans dupliquer la logique d'authentification.

Les composants de routage ont des responsabilités distinctes :

- `RoutePublique` empêche un utilisateur déjà connecté de revenir inutilement vers la connexion ;
- `RouteProtegee` vérifie la présence d'une session valide avant d'afficher une page privée ;
- `RouteAutorisee` contrôle les rôles requis par chaque écran.

Les composants réutilisables `Champ`, `ChampMotDePasse`, `Pagination`, `StatutBadge`, `Toast`, `Avatar` et `Compteur` uniformisent les formulaires, les états visuels et les messages affichés à l'utilisateur. Les pages métier restent ainsi concentrées sur le chargement des données et les actions spécifiques à leur domaine.

Le module `src/api.js` centralise la base URL, les appels Axios et les intercepteurs. En cas de réponse `401`, la session locale est invalidée et l'utilisateur est redirigé vers la connexion. Les erreurs de validation Laravel sont reprises dans les formulaires afin d'indiquer précisément les champs à corriger.

---

5. Travail réalisé sur le backend

5.1 Authentification

Routes publiques :

- `POST /api/connexion`
- `POST /api/creer-compte`
- `GET /api/health`
- `GET /api/openapi.json`

Fonctionnalités :

- Connexion par e-mail et mot de passe.
- Création d'un jeton Sanctum.
- Déconnexion et suppression du jeton courant.

- Consultation de l'utilisateur connecté avec `/me`.
- Changement de mot de passe.
- Limitation du nombre de tentatives de connexion.

L'inscription publique crée exclusivement un compte étudiant et le dossier étudiant associé.

Le flux d'une connexion est le suivant :

- Le frontend envoie l'adresse e-mail et le mot de passe à l'API.
- Laravel vérifie les données et le compte correspondant.
- Sanctum crée un jeton personnel associé à l'utilisateur.
- Le frontend conserve le jeton et les informations minimales de session.
- Chaque appel privé réutilise le jeton dans l'en-tête `Authorization`.
- La déconnexion révoque le jeton courant côté serveur et nettoie la session côté navigateur.

Les mots de passe ne sont jamais enregistrés en clair : le modèle `User` applique le cast Laravel `hashed`. Les journaux d'audit retirent également les champs sensibles tels que le mot de passe, sa confirmation et les jetons.

5.2 API versionnée

Les routes principales sont disponibles sous `/api/v1`. Les anciennes routes `/api/...` sont conservées pour compatibilité.

Ressources disponibles :

- `/api/v1/etudiants`
- `/api/v1/formations`
- `/api/v1/inscriptions`
- `/api/v1/annees-academiques`
- `/api/v1/paiements`
- `/api/v1/presences`
- `/api/v1/notes`
- `/api/v1/documents`
- `/api/v1/notifications`
- `/api/v1/dashboard`
- `/api/v1/utilisateurs`

Les listes utilisent la pagination Laravel. Le paramètre `per_page` accepte une valeur comprise entre 1 et 100.

5.3 Gestion des rôles

Les rôles prévus sont :

- `administrateur`
- `direction`
- `secretaire`
- `enseignant`
- `etudiant`

Fonctionnalité	Administrateur	Direction	Secrétaire	Enseignant	Étudiant
Consulter le tableau de bord	Oui	Oui	Oui	Oui	Non
Consulter les étudiants	Oui	Oui	Oui	Oui	Non

Fonctionnalité	Administrateur	Direction	Secrétaire	Enseignant	Étudiant
Gérer les paiements	Oui	Oui	Oui	Non	Consultation personnelle
Gérer les notes	Oui	Oui	Non	Oui	Consultation personnelle
Gérer les présences	Oui	Oui	Non	Oui	Consultation personnelle
Créer un utilisateur	Oui	Oui	Non	Non	Non
Modifier le rôle d'un utilisateur	Oui	Non	Non	Non	Non
Créer un administrateur	Oui	Non	Non	Non	Non
Créer un directeur	Oui	Non	Non	Non	Non
Modifier son mot de passe	Oui	Oui	Oui	Oui	Oui

La direction peut créer les comptes opérationnels et étudiants, mais ne peut pas créer un administrateur ni un autre compte de direction.

5.4 Gestion des utilisateurs

- Liste des utilisateurs pour l'administrateur et la direction.
- Création des comptes selon la hiérarchie des rôles.
- Modification du nom et de l'adresse e-mail par l'administrateur.
- Modification du rôle par l'administrateur.
- Modification facultative du mot de passe par l'administrateur.
- Conservation du mot de passe existant si aucun nouveau mot de passe n'est fourni.

Les règles de création sont contrôlées dans les Form Requests, avant l'arrivée des données dans les contrôleurs. La direction peut gérer les comptes opérationnels et étudiants, tandis que la création ou la modification des comptes administrateur et direction reste réservée à l'administrateur.

5.5 Modules métier détaillés

- Années académiques : définition d'une année active utilisée par défaut lors d'une inscription.
- Étudiants : création, modification, suppression, recherche et filtrage par formation, avec rattachement facultatif à un compte utilisateur.
- Formations : gestion du catalogue des formations et de leurs informations descriptives.
- Inscriptions : association d'un étudiant à une formation, suivi de la date et du statut, et utilisation automatique de l'année active.
- Paiements : montant, date, référence, statut, notes et notifications adressées à l'administration et à la direction.
- Notes : matière, valeur, barème, coefficient et commentaire ; la valeur ne peut pas dépasser le barème et le bulletin normalise les résultats sur

20.

- Présences : date, statut, retard et motif ; une même inscription ne peut pas recevoir deux présences pour la même date.
- Documents : contrôle du type et de la taille du fichier, stockage sur le disque privé Laravel et téléchargement protégé.
- Notifications : compteur des éléments non lus, consultation, marquage individuel ou global comme lu et rafraîchissement périodique.
- Tableau de bord : agrégation des indicateurs étudiants, formations, inscriptions, paiements et tendances mensuelles.

5.6 Validation, sécurité et traçabilité

Les requêtes d'écriture utilisent des classes dédiées afin de vérifier les types, les formats, les valeurs autorisées, les relations existantes et les confirmations de mot de passe. Les contrôleurs appliquent ensuite les règles de rôle et renvoient des réponses JSON cohérentes.

Les routes privées sont protégées par Sanctum, le middleware d'audit et le middleware de rôles. La pagination borne `per_page` entre 1 et 100 afin d'éviter des réponses excessivement volumineuses. Les mutations sont enregistrées dans `audit_logs`, sans conserver les secrets.

5.7 Règles métier

- Une inscription utilise automatiquement l'année académique active si aucune année n'est transmise.
- Une note ne peut pas dépasser son barème.
- Les notes sont pondérées par coefficient et normalisées sur 20 dans le bulletin.
- Une présence est unique pour une inscription et une date donnée.
- Les paiements utilisent une référence unique facultative.
- Les documents sont conservés sur le disque privé Laravel.
- Les mutations API sont enregistrées dans les journaux d'audit avec les secrets expurgés.
- Les paiements peuvent générer des notifications pour l'administration et la direction.

---

6. Identifiant de démonstration administrateur

Un compte administrateur a été créé pour les démonstrations :

- E-mail : `admin@ecole.local`
- Rôle : `administrateur`
- Mot de passe : `Adm-e02e6cb20550!`

Ce mot de passe est réservé aux démonstrations et doit être modifié après la première connexion.

---

7. Installation locale

7.1 Prérequis

- Git
- Node.js et npm
- PHP 8.3+

- Composer
- MySQL
- Laragon, XAMPP ou un environnement équivalent

7.2 Récupération du projet

```
git clone https://github.com/Armel1617/gestion-ecole.git
cd gestion-ecole
```

7.3 Installation du backend

```
cd backend
composer install
copy .env.example .env
php artisan key:generate
```

Configurer ensuite les paramètres MySQL dans `backend/.env`, puis exécuter :

```
php artisan migrate
php artisan serve
```

L'API sera disponible à l'adresse `http://127.0.0.1:8000`.

7.4 Installation du frontend

Dans un second terminal :

```
cd gestion-ecole
npm install
npm run dev
```

Le frontend utilise l'API `http://127.0.0.1:8000/api/v1`.

MySQL doit être démarré pour charger les données. Apache n'est pas obligatoire lorsque le backend est lancé avec `php artisan serve`.

7.5 Déploiement recommandé

En production, le serveur web doit utiliser `backend/public/` comme racine publique et ne doit jamais exposer les répertoires `app`, `config`, `database` ou `.env`. Le frontend doit être construit avec `npm run build`, puis le contenu de `dist/` doit être servi par le serveur web. Les variables de connexion, la clé Laravel, les paramètres de mail et les secrets Sanctum doivent rester dans l'environnement de déploiement.

---

8. Vérifications réalisées

- Build frontend validé avec `npm run build`.
- Analyse frontend exécutée avec `npm run lint`.
- Syntaxe de `routes/api.php` vérifiée avec `php -l`.
- Routes Laravel versionnées vérifiées avec `php artisan route:list`.
- Routes Notes, Présences et Notifications vérifiées.
- Exclusion des fichiers sensibles : `.env`, `vendor`, `node_modules`, logs et fichiers générés.
- Tests backend PHPUnit couvrant l'authentification, les rôles, les ressources métier, les validations et les notifications.

Les avertissements Oxlint restants concernent principalement l'utilisation de `Date` dans le rendu, un composant de compteur et le composant Toast. Ils ne bloquent pas la compilation.

---

9. Limites et recommandations

- MySQL doit être démarré avant d'utiliser l'application.
- Le mot de passe administrateur temporaire doit être changé après connexion.
- Les secrets de production doivent rester dans `.env` et ne doivent jamais être envoyés sur GitHub.
- En production, le serveur web doit pointer vers `backend/public/`.
- Un worker Laravel doit être configuré pour traiter les notifications en queue.
- Des tests frontend automatisés et des tests de parcours navigateur peuvent être ajoutés dans une prochaine version.

10. Conclusion

Le groupe 6 a réalisé une application complète de gestion scolaire avec une séparation claire entre l'interface React et l'API Laravel. Les fonctionnalités principales de gestion, d'authentification, de contrôle des accès, de suivi scolaire et de notifications sont intégrées. Le projet est disponible sur GitHub avec le frontend à la racine et le backend dans `backend/`.